

Proyectos en ML-neural networks

Consideraciones a tomar en cuenta basados en la experiencia de algunos de los “grandes” de las redes neuronales.

Strait of Magellan · Feb 20, 2025



"Magellis A. Valle"

Preparado por Prof. M.A.Valle

Porqué una estrategia en ML-NN

Suponga que ha entrenado un modelo para reconocer gatos y logra una precisión ($TP/(TP+FP)$) del 90%, pero esto no es suficiente para su aplicación.

¿Qué podemos hacer?

Porqué una estrategia en ML-NN

- * Tal vez podríamos **recopilar más datos**, más datos de entrenamiento,
- * O tal vez recopilar más diversidad de gatos o un **conjunto más diversos de casos negativos**.
- * O tal vez quieras entrenar el algoritmo **durante más tiempo** con descenso de gradiente,
- * O tal vez quieras probar un **algoritmo de optimización** diferente, como el algoritmo de optimización Adam,
- * O tal vez probar una **red más grande o más pequeña**, o tal vez quieras probar el **dropout** o la **regularización L2**,
- * O tal vez quieras cambiar la **arquitectura de la red**, como cambiar las **funciones de activación**, cambiar el número de unidades ocultas, etcétera, etcétera.

Porqué una estrategia en ML-NN

Cuando se intenta mejorar un sistema de aprendizaje profundo, a menudo se tienen muchas ideas o cosas que se podrían probar.

Y el problema es que, si se elige mal, es muy posible que se acabe pasando seis meses avanzando en una dirección para darse cuenta después de seis meses de que no ha servido de nada.

¿No sería ideal disponer de formas rápidas y eficaces de averiguar cuáles de todas estas ideas, y tal vez incluso otras ideas, merecen la pena y cuáles puedes descartar con seguridad?

Ortogonalización



Uno de los retos a la hora de crear NNs es que hay tantas cosas que se pueden probar, muchas cosas que se pueden cambiar. Por ejemplo, hay muchos hiperparámetros que se pueden ajustar.

Aquí la clave es tener claro qué hay que ajustar para intentar conseguir un efecto determinado. Este es un proceso que denominamos **ortogonalización**.

Ortogonalización

Para que un sistema de aprendizaje supervisado funcione bien, normalmente es necesario ajustar los parámetros del sistema para garantizar que:

1. Asegurarse de que, como mínimo, el sistema funciona bien con el **conjunto de entrenamiento**.
2. El rendimiento con el conjunto de entrenamiento debe superar una evaluación de aceptabilidad. Para algunas aplicaciones, esto puede significar alcanzar un rendimiento comparable al de los seres humanos. Sin embargo, esto dependerá de la aplicación.
3. Después de obtener buenos resultados en los conjuntos de entrenamiento, esperemos que esto nos lleve a obtener también buenos resultados en el **conjunto de validación**. Y luego esperamos que esto también funcione bien en el **conjunto de prueba (test)**.

Ortogonalización

4. Esperamos que obtener buenos resultados en el **set test** sobre la **función de costo** lo que se traduzca en un buen rendimiento del sistema en el mundo real.

→ Así que esperamos que esto se traduzca, por ejemplo, en usuarios satisfechos con la aplicación de imágenes de gatos felices.

Volviendo al ejemplo del ajuste del televisor, si la imagen del televisor fuera demasiado ancha o demasiado estrecha, queremos tener un solo mando para ajustarla. No queremos tener que ajustar cuidadosamente 10 botones diferentes, que además afectan a cosas diferentes. Queremos un solo botón (o un conjunto pequeño) que solo afecte al ancho de la imagen del televisor.

Métrica de performance: ¿sólo una?

Digamos que está ajustando hiperparámetros, intentando distintas ideas para algoritmos de aprendizaje, y sólo intentando distintas opciones para su sistema de ML.

→ Veremos que el progreso será más rápido si sólo utilizamos una métrica de evaluación que nos diga rápidamente si la idea actual es mejor o no que la anterior.

Recomendación: Utilizar al menos al inicio, una simple métrica de evaluación para el problema.

Proceso empírico de ML:

IDEA → CÓDIGO → CORRE EXPERIMENTO → SALIDA → REFINAR

Métrica de performance: ¿sólo una?

Supongamos que para el problema de identificar gatos, ha entrenado un clasificador A. Al cambiar algunos hiperparámetros ahora tiene el clasificador B. Comparamos:

Clasificador	Precision	Recall
A	95%	90%
B	98%	85%

Recordatorio:

$$\text{Prec} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Rec} = \text{TP} / \text{P} = \text{TP} / (\text{TP} + \text{FN})$$

Usualmente hay un tradeoff entre precision y recall, el problema es que si A lo hace mejor en recall, el B los hace mejor en precision, entonces cuál elegimos?

Métrica de performance: ¿sólo una?

En vez de utilizar dos números, podemos combinar, por ejemplo el **F1-score**:

$$F1 = 1 / (1/Prec + 1/Rec)$$

En este caso, la media armónica entre Prec y Rec.

En este caso A tiene mejor F1. Asumiendo que F1 es una medida razonable para combinar Prec y Rec, podemos setear A como un mejor clasificador que B.

→ Entonces con set de validación, podemos rápidamente iterar y evaluar varios experimentos.

Clasificador	Prec	Rec	F1
A	95%	90%	92.4%
B	98%	85%	91.0%

Métrica de performance: ¿sólo una?

Imagine que nuestra aplicación de reconocimiento de gatos para LATAM, USA y China.

Clasificador	LATAM	US	China
A	3%	7%	5%
B	5%	6%	10%

Valores de F1 en distintas zonas geográficas

El A logra un 3% de error en las imágenes para LATAM.

Si tuviésemos demasiadas medidas este ejercicio sería complicado.

Métricas de optimización vs cumplimiento

Pero... la verdad es que no es tan fácil combinar todo en sólo un índice.

En esto casos puede ser útil tener una medida de **cumplimiento (satisficing)**, así como una que deseamos **optimizar (por ejemplo F1)**.

Clasificador	Accuracy	Running Time
A	90%	80ms
B	92%	95ms
C	95%	1,500ms

Tal vez Ud. Quiera seleccionar el modelo que maximice Acc pero sujeto a que el tiempo de clasificación de imagen sea menos de 100ms.

Métricas de optimización vs cumplimiento

Por ejemplo:

Considere el dispositivo de control de voz como Amazon Echo, en donde despierta el sistema diciendo “Alexa!”

Ud desea maximizar el accuracy de que el sistema se active cuando digamos Alexa, y no lo haga cuando no haga cuando no se pronuncie.

* ¿Qué probabilidad hay que el dispositivo se active cuando se pronuncia la palabra?

* También nos interesaría saber cuántos falsos positivos hay, ¿con qué frecuencia se activa el dispositivo en forma aleatoria?

Entonces, tal vez deseamos optimizar Accuracy pero con un cierto nivel de precision mínimo para evitar que el sistema se active sin querer.

Distribuciones de los datasets

Set de test y de validación (hold out o development set)

* El val set es usado para hyperparameter search, mientras que el de test es para evaluar del desempeño, con algún conjunto de hiperparámetros que nos dió buenos resultados.

Supongamos:

La aplicación opera en estas regiones, LATAM, USA, Europa, China, Australia, India. ¿cómo seteamos el val set y el set de test?

→ podríamos elegir 3 regiones al azar para val set y las otras 3 para test.

mala idea!!

El conjunto de val y test provienen de pdf's distintas!

Distribuciones de los datasets

Set de test y de validación (hold out o development set)

* El val set es usado para hyperparameter search, mientras que el de test es para evaluar del desempeño, con algún conjunto de hiperparámetros que nos dió buenos resultados.

Supongamos:

La aplicación opera en estas regiones, LATAM, USA, Europa, China, Australia, India. ¿cómo seteamos el val set y el set de test?

→ Tenemos que buscar la manera de que las pdf's sean iguales

Tomar todos los datos mezclados aleatoriamente en el val y test set!

De este modo, el set de val y test tendrán de las 6 regiones y ambos sets procederán de la misma pdf.

Distribuciones de los datasets

Set de test y de validación (hold out o development set)

El ejemplo de los préstamos:

Imagine un equipo de ML que dedicó varios meses a optimizar un modelo con el set de validación compuesto por **aprobaciones de préstamos basado en las direcciones geográficas en zonas de ingresos medios**.

El problema específico de aprendizaje automático era: «Dada una entrada X sobre una solicitud de préstamo, ¿se puede predecir y, es decir, si devolverán o no el préstamo?».

Esto ayuda a decidir si se aprueba o no un préstamo. El set de validación procedía de solicitudes de préstamos. Procedían de información geográfica (direcciones) de ingresos medios.

Después de trabajar en esto durante unos meses, el equipo decidió de repente probarlo con **datos de info geográfica de ingresos bajos**.

Distribuciones de los datasets

Set de test y de validación (hold out o development set)

El ejemplo de los préstamos (continuación): **¿qué pasó?**

Los datos de distribución para zonas geográficas de ingresos medios y bajos son muy diferentes. **Y en tanto, todo el tiempo que invirtieron en el primer caso, simplemente no funcionó bien en el segundo.**

Así que este equipo en concreto perdió unos tres meses de tiempo y tuvo que volver atrás y rehacer gran parte del trabajo.

Lo que ocurrió aquí fue que el equipo pasó tres meses centrado en un objetivo y, al cabo de ese tiempo, el director preguntó: «¿Cómo van con el otro objetivo?».

Se trataba de una ubicación totalmente diferente. Fue una experiencia muy frustrante para el equipo.

¿cuál es la enseñanza de esta historia?

Distribuciones de los datasets

Set de test y de validación (hold out o development set)

Recuerde: El val set y el test set deben venir de la misma distribución. La forma que selecciona el training set afectará cuán bien va a hacer blanco en el target.

Para configurar un set de validación y un set de test es elegir tal que ambos reflejen los datos que se espera obtener en el futuro y que se consideren importantes para obtener buenos resultados.

Tamaño de los sets de Val y Test

- * La regla general del **70/30**: 70 para train y el otro 30 para test. O también la regla del **60/20/20**: 60 para train, 20 para test y 20 para val.
 - muy útil para set de datos más pequeños (de más de 1,000 instancias)
- * Si tenemos por ejemplo, mas de 1e6 instancias, podemos destinar mucho más para el entrenamiento (sobre 80%) y dejar el otro 20% para test y val.
- * Recordemos que el test set es para evaluar el modelo y así se sabe qué tan bien lo hace después de haber entrenado.... entonces no necesitamos millones de ejemplos para evaluar, y con tan sólo 10,000 puede ser suficiente para tener confianza en los estimadores.

¿Cuándo cambiar las métricas?

Supongamos que tenemos dos clasificadores entrenados para reconocer gatos y la métrica que utilizamos para evaluar es el error de clasificación:

MODELO	Métrica error
A	3%
B	5%

* El A sería mejor, pero al ensayar parece ser que el A por algún motivo deja pasar imágenes de animales muertos. Esto sería inaceptable.

* En cambio el B, tiene más error pero no tiene el problema de A.

→ Desde el punto de vista de la empresa, y de los usuarios, el B es mejor.

¿Cuándo can

El error de misclassifica

$$error = \frac{1}{n} \sum_{i=1}^n \mathbf{1}_{\{y_{pred}^{(i)} \neq y^{(i)}\}}$$

Cuando pasa esto, la métrica de evaluación ya no clasifica correctamente las **preferencias entre modelos**.

→ Señal de que tenemos que cambiar de métrica o tal vez el conjunto de test.

¿Cuándo c

Vemos que la métrica
MUERTO por igual, l

Entonces podríamos
imágenes de animal

$$error = \frac{1}{\sum w_i} \sum_{i=1}^n \mathbf{1}\{y_{pred}^{(i)} \neq y^{(i)}\} w_i$$

Entonces de esta forma nuestra métrica de evaluación capturaría mejor qué algoritmo lo hace mejor. **No insistamos en una medida de desempeño si no captura el desempeño de lo que queremos.**

¿Cuándo cambiar las métricas?

* El Task en ML-NN es

Primero: definir el objetivo y definir la métrica.

Segundo: ajustar las “perillas” para colocar el objetivo como un problema separado.

El primero y segundo paso son pasos separados!

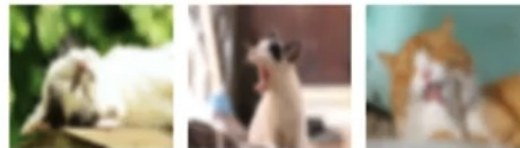
¿Cuándo cambiar las métricas?

* Sigamos con el ejemplo del modelo A y B.

Test set



User images



Imagine que cuando implemente su producto algorítmico, descubra que el algoritmo B parece funcionar mejor, a pesar de que el otro funciona mejor en su conjunto de test. Ud. Piensa que ha estado entrenando con imágenes de muy buena calidad descargadas de Internet, pero cuando la implementa en la aplicación móvil, los usuarios suben todo tipo de imágenes mucho menos encuadradas, en las que no solo aparece el gato, sino que los gatos tienen expresiones faciales divertidas, quizá las imágenes son mucho más borrosas, y cuando prueba sus modelos, descubre que el algoritmo B en realidad funciona mejor. Así que este sería otro ejemplo de cómo sus conjuntos de métricas y/o pruebas de test/validación fallan.

¿Cuándo cambiar las métricas?

* ¿Cuál es el problema aquí?

.....

Moraleja:

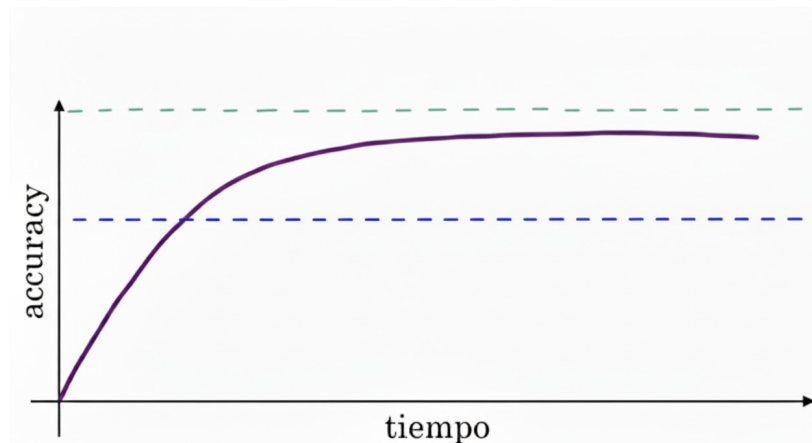
Si obtiene buenos resultados en su métrica y en la distribución actual de sus conjuntos de val y/o test, pero eso no se corresponde con buenos resultados en la aplicación que realmente le interesa, entonces cambie su métrica y/o su conjunto de prueba de desarrollo.

¿Human-Level performance?

Opcional

* Los avances en DL y los algoritmos en ML son cada vez más eficientes y más factibles en áreas de aplicación en donde los humanos tradicionalmente son muy competentes.

Comparación con el desempeño humano



Bayes optimal error

Human performance

- * En proyectos de ML, el desempeño avanza rápido mientras el equipo no supera el desempeño humano.
- * Conforme el algoritmo supera al humano, el progreso se torna lento.
- * La esperanza es lograr aproximarse lo más posible al “**Bayes optimal error**”.

¿Human-Level performance?

Opcional

Pero ... ¿qué es Bayes error?

*The **ideal model** is an oracle that simply knows the true probability distribution that generates the data. Even such a model will still incur some error on many problems, because there may still be some **noise in the distribution**. In the case of supervised learning, the mapping from x to y may be inherently stochastic, or y may be a deterministic function that involves other variables besides those included in x . The error incurred by an oracle making predictions from the true distribution $p(x, y)$ is called the **Bayes error**.*

Avoidable Bias

Necesario recordar: *bias-variance tradeoff*

Supongamos que hemos estado usando nuestra app de clasificación de gatos. Digamos que:

	Tasa error
Humanos	1%
Training error	8%
Test error	10%

Entonces tenemos una gran brecha entre lo que hace el modelo y los humanos. Entonces trataríamos de mejorar el training error, reducir el bias, quizás con una red más grande, o más epochs..

Avoidable Bias

Necesario recordar: *bias-variance tradeoff*

Pero ahora supongamos que los **humanos tiene una tasa de error del 7.5%** para identificar gatos. Las imágenes son borrosas, a si que la cosa es difícil.

	Tasa error
Humanos	7.5%
Training error	8%
Test error	10%

En este caso, aunque el error de entrenamiento y error de test sean los mismos que en el otro ejemplo, verá que quizá en realidad lo está haciendo muy bien con el conjunto de entrenamiento. Solo está rindiendo un poco peor que el nivel humano.

Entonces en este caso, quizás deseamos reducir la varianza, intentando poner regularización o consiguiendo más data.

Nota: Por definición, el error a nivel humano es peor que el error de Bayes, ya que nada puede ser mejor que el error de Bayes, pero el error a nivel humano podría no estar muy lejos del error de Bayes. **Aquí ya no tendríamos mucho más por hacer para reducir el error.**

Avoidable Bias

Necesario recordar: *bias-variance tradeoff*

Definición:

Avoidable Bias = Bayes error (o su aprox) – training error

Entonces, lo que queremos es **seguir mejorando el rendimiento de training hasta llegar al error de Bayes**, pero en realidad Ud. no quiere superar el error de Bayes. *Equivale a una medida de cuánta varianza tenemos en el algoritmo de aprendizaje.*

Ud. no puede superar el error de Bayes a menos que esté sobreajustando. Y esto, *la diferencia de error entre training y test, sigue siendo una medida del problema de varianza de su modelo.*

Así en el ejemplo anterior diríamos que el avoidable Bias es de 0.5%, mientras que 2% es una medida de varianza que podemos mejorar.

Avoidable Bias

Necesario recordar: *bias-variance tradeoff*

Así del ejemplo anterior diríamos:

“que el avoidable Bias es de 0.5%, mientras que 2% es una medida de varianza que podemos mejorar”.

Y del ejemplo inicial en contraste, diríamos:

“tenemos un avoidable Bias del 7%, mientras que 3% es una medida de cuánta varianza tenemos”,

...por lo que en este caso tenemos más potencial en reducir el Avoidable Bias.

Más de Human Level Performance

Opcional

Recordemos que “*human-level error*” equivale a decir una forma de estimar el **Bayes error**.

Supongamos el problema de clasificación de imágenes médicas:

(a) Humano típico 3% error

(b) Médico humano típico1% error

(c) Reunión de médicos0.5% error

Entonces, ¿cuál sería el human-level error? 1%, 1%, 0.5%??

Más de Human Level Performance

Opcional

Si se quiere una aproximación o una estimación del error de Bayes, dado que un equipo de médicos experimentados que discuten y debaten puede alcanzar un error del 0.5%, sabemos que el error de Bayes es inferior o igual al 0.5%

→ Por lo tanto, lo que haría en este contexto es utilizar el 0.5% como nuestra estimación del error de Bayes. Así que definiría el rendimiento a nivel humano como el 0.5%

*** Conclusión:**

Hay que tener claro cuál es el propósito al definir el término «error a nivel humano». Y si se trata de demostrar que se puede superar a un solo ser humano y, por lo tanto, argumentar a favor de la implementación del sistema en algún contexto, tal vez esta sea la definición adecuada.

Más de Human Level Performance

Opcional

Siguiendo con el ejemplo:

Supongamos, para un ejemplo de diagnóstico por imágenes médicas, que su error de:

- * entrenamiento es del 5% y $\leftarrow \text{Train error} - \text{test error} = \text{variance} = 1\%$
- * error test es del 6 %.
- * Bayes error : tomemos 1% (rendimiento a nivel humano)

→ sesgo evitable (**avoidable Bias**) = $|\text{Bayes Error} - \text{Train Error}| = 4\%$

Vemos que esto es claramente más importante que el problema de la varianza. Por lo tanto, en este caso, debería enfocarse en **técnicas de reducción del sesgo, como entrenar una red más grande**. Veamos ahora un segundo ejemplo. Supongamos que tu error de entrenamiento es del 1 % y tu error de desarrollo es del 5 %.

Más de Human Level Performance

Opcional

Veamos ahora un segundo ejemplo. Supongamos que:

- * train error es 1%
- * test error es 5% ← $\text{Train error} - \text{test error} = \text{variance} = 4\%$
- * Bayes error : tomemos entre 0.5% y 1% (rendimiento a nivel humano)

En este caso, veremos que el **avoidable bias** estará entre 0 y 0.5%.

Entonces, aquí sería mejor focalizarse en técnicas de **reducción de la varianza**, como la **regularización** o la **obtención de un conjunto de entrenamiento más grande**.

Más de Human Level Performance

Opcional

Veamos ahora un tercer ejemplo. Supongamos que:

- * train error es 0.7%
- * test error es 0.8% ← $\text{Train error} - \text{test error} = \text{variance} = 0.1\%$
- * Bayes error : tomemos entre 0.5% (rendimiento a nivel humano)

En este caso, veremos que el **avoidable bias** estará en 0.2%.

Aquí **SÍ** importa que error humano tomamos como Bayes error.

Como vemos, el problema se vuelve complicado de analizar conforme logramos un desempeño cercano al del humano.

Más de Human Level Performance

Opcional

Para RESUMIR:

1. Cuando tenemos una estimación del error humano para una tarea que los humanos pueden hacer bastante bien, podemos usar el human-level performance como un indicador o una aproximación del Bayes error.

** La diferencia entre Bayes error y Train error dice cuánto avoidable bias hay.*

** La diferencia entre Train error y Test error indica cuán problema de varianza hay.*

Esto le permite tomar decisiones más rápidamente sobre si debe centrarse en intentar reducir un sesgo o en intentar reducir la varianza de su algoritmo.

Estas técnicas suelen funcionar bien hasta que se supera el rendimiento humano, momento en el que es posible que ya no se disponga de una buena estimación del error bayesiano que le ayude a tomar esta decisión con claridad.

Superando el Human Level Performance

Opcional

Supongamos los siguiente errores para una tarea:

* Team de humanos	0.5%	← Train error – test error = variance = 0.2%
* Un humano	1%	
* Train error	0.6%	← Avoidable Bias = 0.1%
* Test error.....	0.8	

Cuál es el **avoidable bias**?

---- con estos resultados, quizás nos concentramos en reducir la varianza.

Superando el Human Level Performance

Opcional

Supongamos caso más complicado con los siguiente errores para una tarea:

* Team de humanos	0.5%	← Train error – test error = variance = 0.1%
* Un humano	1%	
* Train error	0.3%	← Avoidable Bias = ?? 0.1% o 0.2% o que?
* Test error.....	0.4%	

Cuál es el **avoidable bias**?

---- con estos resultados, quizás nos concentramos en reducir la varianza, o mejor en el avoidable bias???

El problema del aprendizaje automático es simplemente menos claro. Esto no significa que no se pueda avanzar, es posible que aún se puedan lograr avances significativos, pero algunas de las herramientas que se tienen para indicar una dirección clara simplemente no funcionan tan bien.

Majora del desempeño del modelo

Opcional

Para que un algoritmo de aprendizaje supervisado funcione bien tiene que:

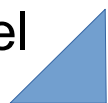
1. Fitear bastante bien en el set de training (en otras palabras, que el **avoidable bias sea bajo**)
2. El performance sobre el test set debe generalizar bastante bien (o sea, la **varianza** no debe ser alta).

Majora del desempeño del modelo

Opcional

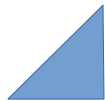
Recomendaciones para un proyecto de aprendizaje supervisado:

* Human-level



← Monitorear Avoidable Bias (entrenar model más grande, mejor algoritmo de optimización (momentum, RMSProp, Adam), buscar otros hiperparámetros o arquitecturas)

* train error



← Monitorear Variance (más datos, regularizador (L2, dropout), buscar otros hiperparámetros o arquitectura)

* test error